Java Backend Architecture

Interview Series

Chapter 13.3
Spring WebFlux & Non-Blocking IO

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Introduction

Spring WebFlux is the reactive web framework introduced in Spring 5, built on top of Project Reactor and the Reactive Streams specification. Unlike Spring MVC which requires Servlet API and blocking I/O, WebFlux can run on both Netty (default, non-blocking event loop) and Servlet 3.1+ containers like Tomcat in non-blocking mode. WebFlux offers two programming models: the familiar annotation-based (@RestController, @GetMapping) and a functional model (RouterFunction + HandlerFunction) that treats routing as composable function chains. The DispatcherHandler is WebFlux's equivalent of DispatcherServlet.

The real power of WebFlux emerges through its ecosystem: WebClient replaces RestTemplate with a fully non-blocking HTTP client, R2DBC provides reactive relational database access, reactive MongoDB and Cassandra drivers stream results as Flux, and reactive Spring Security propagates security context without ThreadLocal. Server-Sent Events (SSE) and reactive WebSocket support make streaming APIs straightforward. However, WebFlux is not universally appropriate — its complexity is only justified when the application is genuinely I/O-bound with high concurrency, the team understands reactive programming, and all dependencies in the chain support non-blocking operation.

Key Concepts & Definitions

WebFlux	Spring reactive web framework built on Reactor; supports both Netty (default) and Servlet 3.1+ containers.
DispatcherHandler	WebFlux central request handler; analogous to DispatcherServlet in Spring MVC.
RouterFunction	Functional routing in WebFlux: RouterFunctions.route(predicate, handlerFn) composes routes functionally.
HandlerFunction	Functional endpoint: takes ServerRequest, returns Mono.
WebClient	Non-blocking, reactive HTTP client replacing RestTemplate; returns Mono/Flux for all responses.
R2DBC	Reactive Relational Database Connectivity; non-blocking drivers for PostgreSQL, MySQL, H2, etc.
ServerRequest	Functional WebFlux equivalent of HttpServletRequest; provides reactive body access.
ServerResponse	Functional WebFlux response builder; created via ServerResponse.ok().bodyValue(...).
WebFilter	Reactive equivalent of Servlet Filter; takes ServerWebExchange and WebFilterChain, returns Mono<Void>.
SSE	Server-Sent Events; WebFlux supports returning Flux<ServerSentEvent> for server-push streaming.
Netty EventLoop	Boss EventLoopGroup accepts connections; Worker EventLoopGroup handles I/O — tiny fixed pool, never block.
Reactive Security	Spring Security reactive module propagates SecurityContext via Reactor Context (not ThreadLocal).

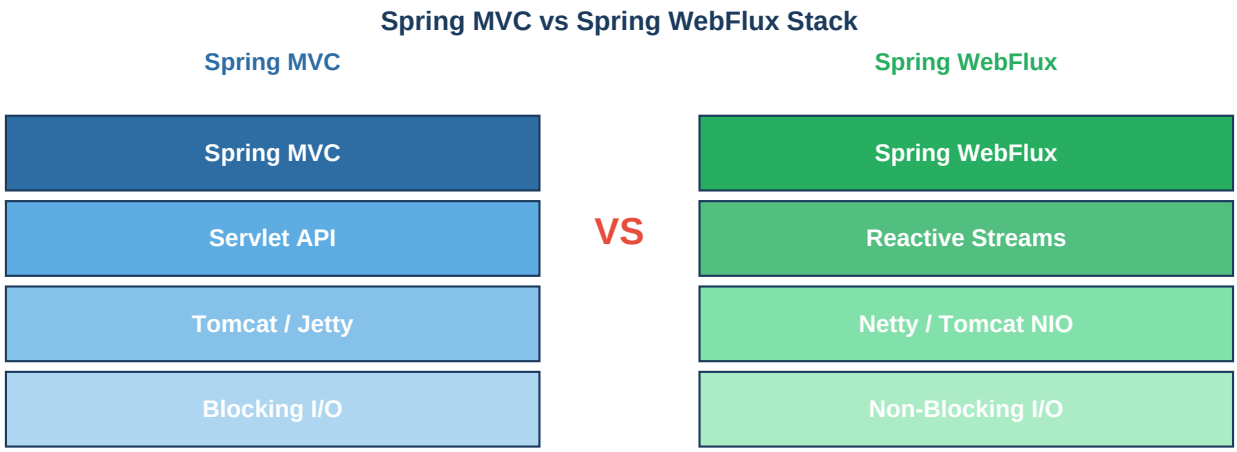
Reactive MongoDB	Spring Data Reactive MongoDB returns Mono/Flux; uses the async MongoDB driver under the hood.
Functional Endpoint	Route handler defined as a lambda/method reference — more explicit, testable, and type-safe than annotations.
application.properties	Set spring.main.web-application-type=reactive and spring.threads.virtual.enabled for reactive config.

WebFlux vs Spring MVC Comparison

Feature	Spring MVC	Spring WebFlux
Server	Tomcat/Jetty (Servlet)	Netty (default) or Servlet NIO
Threading	Thread-per-request	Event loop (small fixed pool)
Return types	Object, ResponseEntity	Mono<T>, Flux<T>
DB Access	JDBC (blocking)	R2DBC (non-blocking)
HTTP Client	RestTemplate (blocking)	WebClient (non-blocking)
Security Context	ThreadLocal	Reactor Context
Testing	MockMvc	WebTestClient
Use case	Low-medium concurrency	High concurrency I/O
Complexity	Low	High
Virtual Threads	Java 21 + Loom (viable)	Not needed if using Loom

Architecture Diagrams

Spring MVC vs Spring WebFlux Stack



WebClient Non-Blocking HTTP Pipeline

WebClient Non-Blocking HTTP Pipeline

Entire chain is non-blocking — no thread held while waiting for HTTP response

Routing Models Comparison

WebFlux Routing: @RestController vs RouterFunction

Annotated (@RestController)

```
@GetMapping("/users")

Returns Flux<User>

DispatcherHandler dispatches

@RequestBody → Mono<Req>
```

Functional (RouterFunction)

```
RouterFunctions.route()

RequestPredicate

HandlerFunction<R>

ServerRequest / Response
```

Both styles co-exist; DispatcherHandler routes both

Java Code Examples

1. Spring WebFlux @RestController

```
@RestController @RequestMapping("/api/users") public class UserController { private final UserRepository repo; // R2DBC reactive repository @GetMapping public Flux<User> getAllUsers() { return repo.findAll() .filter(User::isActive) .take(100); } @GetMapping("/{id}") public Mono<ResponseEntity&lt;User&gt;> getUser(@PathVariable Long id) { return repo.findById(id) .map(ResponseEntity::ok) .defaultIfEmpty(ResponseEntity.notFound().build()); } @PostMapping @ResponseStatus(HttpStatus.CREATED) public Mono&lt;User&gt; createUser(@RequestBody Mono<UserRequest> req) { return req.map(r -> new User(r.getName(), r.getEmail())) .flatMap(repo::save); } }
```

2. Functional RouterFunction Endpoint

```
@Configuration public class UserRouter { @Bean public RouterFunction<ServerResponse> routes(UserHandler handler) { return RouterFunctions.route() .GET("/api/users", handler::listUsers) .GET("/api/users/{id}", handler::getUser) .POST("/api/users", handler::createUser) .filter((req, next) -> { log.info("Request: {} {}", req.method(), req.path()); return next.handle(req); }) .build(); } } @Component public class UserHandler { public Mono<ServerResponse> listUsers(ServerRequest req) { return ServerResponse.ok() .contentType(MediaType.APPLICATION_JSON) .body(repo.findAll(), User.class); } }
```

3. WebClient Non-Blocking HTTP Call

```
@Service public class PaymentService { private final WebClient webClient; public PaymentService(WebClient.Builder builder) { this.webClient = builder .baseUrl("https://payment-api.example.com") .defaultHeader(HttpHeaders.CONTENT_TYPE, "application/json") .build(); } public Mono<PaymentResult> processPayment(PaymentRequest req) { return webClient.post() .uri("/payments") .bodyValue(req) .retrieve()
```

```
.onStatus(HttpStatus::is4xxClientError, resp ->
resp.bodyToMono(String.class).map(PaymentException::new)) .bodyToMono(PaymentResult.class)
.timeout(Duration.ofSeconds(10)) .retryWhen(Retry.backoff(2, Duration.ofMillis(200))); } }
```

4. R2DBC Reactive Repository

```
// pom: spring-boot-starter-data-r2dbc, r2dbc-postgresql @Repository public interface
OrderRepository extends ReactiveCrudRepository<Order, Long> { Flux<Order> findByUserId(Long
userId); Flux<Order> findByStatusAndCreatedAtAfter(OrderStatus status, LocalDateTime since);
@Query("SELECT * FROM orders WHERE total > :amount ORDER BY created_at DESC") Flux<Order>
findHighValueOrders(@Param("amount") BigDecimal amount); } // application.yml //
spring.r2dbc.url: r2dbc:postgresql://localhost:5432/mydb // spring.r2dbc.username: postgres //
spring.r2dbc.password: secret
```

5. Server-Sent Events (SSE) Streaming

```
@RestController @RequestMapping("/api/events") public class EventStreamController { private
final EventService eventService; @GetMapping(produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<ServerSentEvent<String>> streamEvents() { return eventService.getEventStream()
.map(event -> ServerSentEvent.<String>builder() .id(String.valueOf(event.getId()))
.event(event.getType()) .data(event.getPayload()) .build()) .doOnError(err -> log.error("SSE
error", err)); } } // Client: const es = new EventSource("/api/events"); es.onmessage = ...
```

6. WebTestClient Integration Test

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT) class
UserControllerTest { @Autowired WebTestClient webClient; @Test void shouldReturnUsers() {
webClient.get().uri("/api/users") .accept(MediaType.APPLICATION_JSON) .exchange()
.expectStatus().isOk() .expectHeader().contentType(MediaType.APPLICATION_JSON)
.expectBodyList(User.class) .hasSize(3) .value(users ->
assertThat(users).allMatch(User::isActive)); } @Test void shouldReturn404ForMissingUser() {
webClient.get().uri("/api/users/999") .exchange() .expectStatus().isNotFound(); } }
```

Interview Q&A;

Q1. What is Spring WebFlux and how does it differ from Spring MVC?

Spring WebFlux is the reactive web framework in Spring 5+ built on Project Reactor and Reactive Streams. Unlike Spring MVC (Servlet API, blocking I/O, thread-per-request), WebFlux uses non-blocking I/O on Netty's event loop, returns Mono/Flux from controllers, and uses WebClient instead of RestTemplate. Both support the same annotation model, but MVC blocks threads while WebFlux does not.

Q2. Can Spring WebFlux run on Tomcat?

Yes. WebFlux can run on any Servlet 3.1+ container (Tomcat, Jetty) in addition to Netty. On Tomcat it uses non-blocking NIO. However, Netty is the default and recommended server for WebFlux because it was designed as an NIO event-loop server. On Servlet containers, each request still occupies a Servlet thread during dispatch, reducing the throughput benefit.

Q3. What is the difference between RouterFunction and @RestController in WebFlux?

@RestController uses Spring's annotation-driven model: methods annotated with @GetMapping etc. are discovered by DispatcherHandler. RouterFunction is the functional model: routes are defined as composable RouterFunctions beans using a fluent DSL. Both coexist in the same application. Functional endpoints are more explicit and easier to unit-test without Spring context.

Q4. What is WebClient and why does it replace RestTemplate?

WebClient is a non-blocking, reactive HTTP client that returns Mono<T> or Flux<T>. RestTemplate is synchronous and blocking — it holds a thread for the entire HTTP call duration. Using RestTemplate in a

WebFlux application blocks the Netty event loop thread, destroying concurrency. WebClient integrates naturally with reactive pipelines and supports streaming responses.

Q5. What is R2DBC and how does it differ from JDBC?

R2DBC (Reactive Relational Database Connectivity) provides non-blocking, reactive drivers for relational databases (PostgreSQL, MySQL, H2, MSSQL). Unlike JDBC which uses blocking socket calls (holding threads), R2DBC uses event-driven, non-blocking I/O. Spring Data R2DBC provides ReactiveCrudRepository with Flux/Mono return types, allowing fully reactive database access in WebFlux applications.

Q6. What is DispatcherHandler in WebFlux?

DispatcherHandler is the central request processor in WebFlux, analogous to DispatcherServlet in Spring MVC. It discovers HandlerMapping, HandlerAdapter, and HandlerResultHandler beans to route requests to handlers and process results. It processes requests reactively, returning Mono<Void> for each request, and works with both annotated controllers and RouterFunction beans.

Q7. How does Spring Security work in a reactive application?

Spring Security Reactive uses the Reactor Context instead of ThreadLocal to propagate SecurityContext. It provides ReactiveSecurityContextHolder for reading the context. WebFilter-based security chain (SecurityWebFilterChain) processes requests reactively. Authentication uses ReactiveAuthenticationManager; authorization uses ReactiveAuthorizationManager. @PreAuthorize still works with reactive return types.

Q8. What is a WebFilter in WebFlux?

WebFilter is the reactive equivalent of a Servlet Filter. It intercepts ServerWebExchange objects (encapsulating request + response) and calls chain.filter(exchange) to continue. Use cases: authentication, logging, CORS, rate limiting, request ID injection. Filters return Mono<Void> and are composable. They are ordered and applied before routing to a handler.

Q9. How do you implement Server-Sent Events (SSE) in WebFlux?

Return Flux> (or just Flux<T>) from a controller with produces = MediaType.TEXT_EVENT_STREAM_VALUE. WebFlux streams items as SSE messages without holding threads between events. Each item in the Flux becomes an SSE event with id, event type, and data fields. The connection stays open until the Flux completes or the client disconnects.

Q10. When should you NOT use Spring WebFlux?

Do not use WebFlux when: (1) the team lacks reactive programming experience — debugging is hard; (2) any layer in the stack uses blocking I/O (JDBC, blocking libraries) and cannot be replaced; (3) the application has low concurrency where MVC + virtual threads is simpler; (4) frameworks/libraries you depend on are not reactive-compatible. The benefit only materialises for high-concurrency, I/O-heavy workloads.

Q11. How do you test WebFlux controllers?

Use WebTestClient — a non-blocking, fluent test client for WebFlux. For unit tests, bind directly to the controller: WebTestClient.bindToController(new UserController(...)). For integration tests, use @SpringBootTest with WebEnvironment.RANDOM_PORT and @Autowired WebTestClient. It supports .expectStatus(), .expectBody(), .expectBodyList(), and reactive stream assertions.

Q12. What is the Netty event loop and why must you never block in it?

Netty uses a small fixed pool of EventLoop threads (boss + worker groups, typically 2x CPU count). Each EventLoop thread handles I/O events for hundreds/thousands of connections. If you block an EventLoop thread (JDBC call, Thread.sleep, etc.) all connections assigned to it stall. Always offload blocking work to Schedulers.boundedElastic() before it reaches the event loop.

Q13. How does context propagation work in WebFlux for things like MDC logging?

WebFlux uses Reactor's Context (a per-subscription immutable map) instead of ThreadLocal. For MDC logging, use a WebFilter to populate Context with trace IDs, then use .contextWrite() in your reactive chain. Libraries like Micrometer Tracing and Spring's ContextPropagation API (Spring 6) handle context propagation automatically across threads and reactive boundaries.

Q14. What is the difference between WebTestClient and MockMvc?

MockMvc simulates Spring MVC requests in-memory (no real network, no actual server) and is synchronous. WebTestClient is reactive and can bind to a WebFlux controller (no server), a RouterFunction, or a running server (real HTTP). It supports both MVC (with WebTestClient.bindToApplicationContext) and WebFlux. For WebFlux, WebTestClient is the standard testing tool.

Q15. How do you configure timeouts and retry in WebClient?

Use .timeout(Duration) on the Mono/Flux returned by WebClient to set response timeout. For connection/read timeouts configure the underlying Netty HttpClient: HttpClient.create().responseTimeout(Duration.ofSeconds(5)). Add retry with .retryWhen(Retry.backoff(n, firstDelay)). For circuit breaking, integrate Resilience4j reactive operators into the WebClient pipeline.

Q16. What reactive MongoDB operations does Spring Data support?

Spring Data Reactive MongoDB provides ReactiveMongoRepository with Flux<T> findAll(), Mono<T> findById(), Mono<T> save(). Custom queries use @Query or ReactiveMongoTemplate. Aggregation pipelines return Flux. Change streams (real-time document change notification) return Flux>. The driver uses the async MongoDB Java driver internally.

Q17. How does WebFlux handle WebSocket connections?

WebFlux supports reactive WebSocket via WebSocketHandler interface: handle(WebSocketSession) returns Mono<Void>. Each session provides Flux for inbound messages and a sink for outbound. Use session.receive().flatMap(msg -> process(msg)).then() pattern. Register via WebSocketHandlerAdapter and SimpleUrlHandlerMapping beans.

Q18. What is defaultIfEmpty and switchIfEmpty in WebFlux context?

defaultIfEmpty(value) emits a static default value when the upstream Mono/Flux completes empty. switchIfEmpty(alternativePublisher) subscribes to an alternative reactive publisher when the upstream is empty — suitable for async fallback (e.g., cache miss → DB lookup). In WebFlux controllers, these are used to return 404 responses when entities are not found.

Q19. How do you configure connection pooling for R2DBC?

Use r2dbc-pool (io.r2dbc:r2dbc-pool). Configure via R2dbcAutoConfiguration or manually: ConnectionFactory pool = ConnectionPoolConfiguration.builder(connectionFactory).initialSize(5).maxSize(20).maxIdleTime(Duration.ofMinutes(30)).build(); Register as a @Bean. Spring Boot auto-configures the pool from spring.r2dbc.pool.* properties.

Q20. How does Spring WebFlux compare to Spring MVC with virtual threads (Java 21)?

With Java 21 virtual threads (spring.threads.virtual.enabled=true), Spring MVC can handle as many concurrent requests as WebFlux without reactive complexity, because blocking a virtual thread costs almost nothing. For purely HTTP-bound workloads, MVC+virtual threads is simpler. WebFlux still wins for streaming (SSE, WebSocket), reactive backpressure, and integration with fully reactive stacks (R2DBC, reactive MongoDB).